Universidade Federal da Paraíba
Centro de Informática
Departamento de Informática

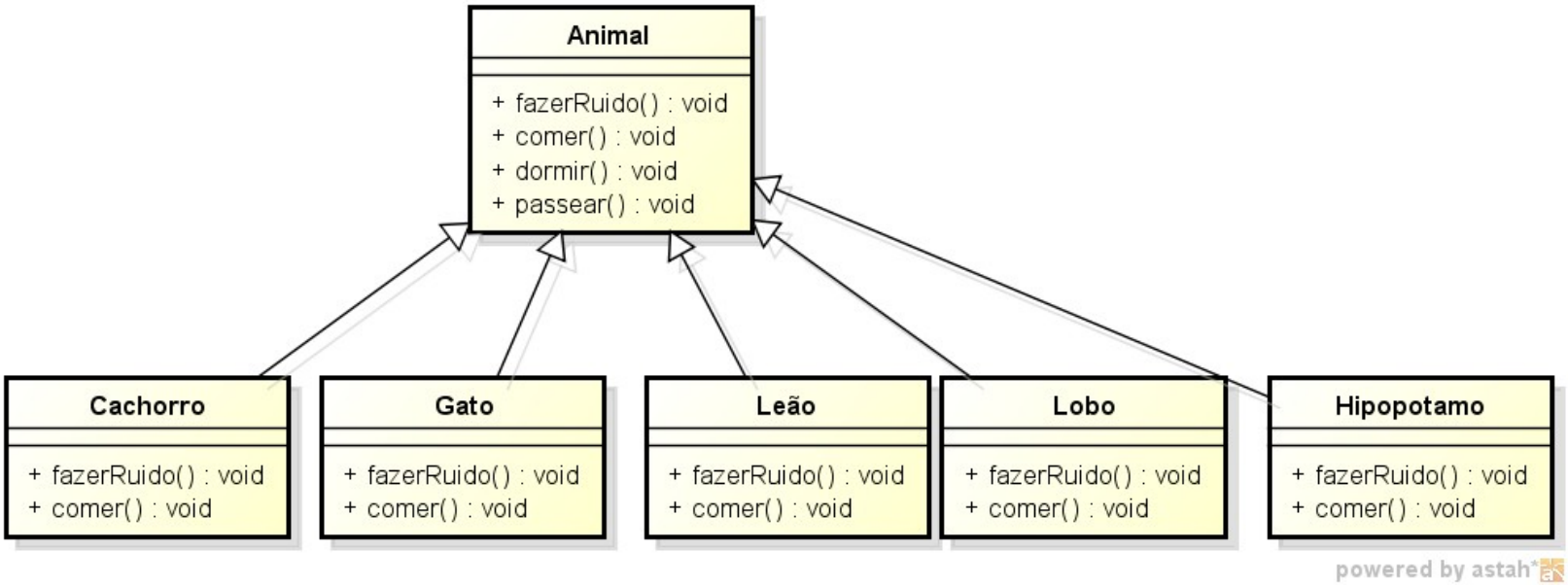
Linguagem de Programação
Polimorfismo

- Derzu Omaia
- derzu@ci.ufpb.br

Motivação

- Com a **herança** podemos evitar códigos duplicados
  - Ex: `Conversivel` não replica códigos de `Veiculo`

- Inserimos um código comum em uma superclasse e deixamos as subclasses herdarem esse código.





Motivação

- Ou seja, estamos definindo um protocolo comum para um grupo de classes...



- Todos os subtipos de `Animais` (`Cachorro`, `Gato`, `Lobo`, etc.) poderão fazer as mesmas coisas...
  - `comer()`, `fazerRuido()`, `dormir()`, `circular()`...



Motivação

- Exemplo:

```
Animal *animal1 = new Animal();
animal1->comer();
animal1->fazerRuido();

Cachorro *cachorro1 = new Cachorro();
cachorro1->comer();
cachorro1->fazerRuido();

Gato *gato1 = new Gato();
gato1->comer();
gato1->fazerRuido();

Lobo *lobo1 = new Lobo();
lobo1->comer();
lobo->fazerRuido();
```



Motivação

- Exemplo:

```
Animal *animal1 = new Animal();
animal1->comer();
animal1->fazerRuido();

Cachorro *cachorro1 = new Cachorro();
cachorro1->comer();
cachorro1->fazerRuido();

Gato *gato1 = new Gato();
gato1->comer();
gato1->fazerRuido();

Lobo *lobo1 = new Lobo();
lobo1->comer();
lobo->fazerRuido();
```

Repetição de código



Motivação

- Lembrando como funciona a declaração e criação de objetos

1. Declaração da variável

```
Cachorro *cachorro1 = new Cachorro();
```

Declara a variável cachorro1

Ponteiro Cachorro



cachorro1



Motivação

- Lembrando como funciona a declaração e criação de objetos

2. Criação do Objeto (Instanciação)

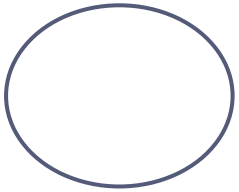
```
Cachorro *cachorro1 = new Cachorro();
```

Aloca espaço para um objeto do tipo Cachorro

Ponteiro Cachorro



cachorro1



Objeto Cachorro



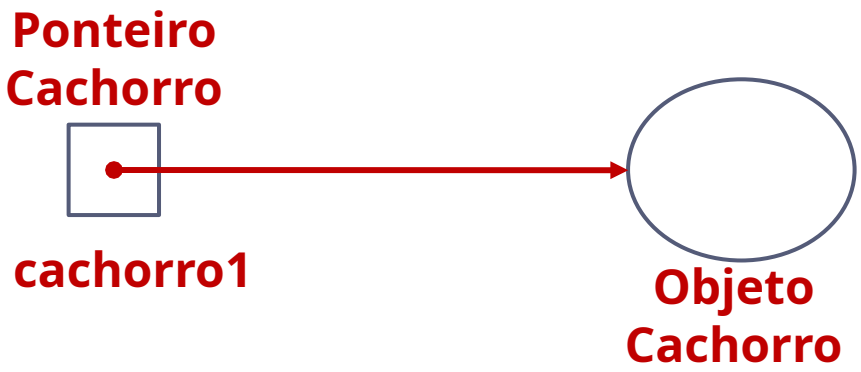
Motivação

- Lembrando como funciona a declaração e criação de objetos

3. Vincula o objeto a variável de referência

```
Cachorro *cachorro1 = new Cachorro();
```

Atribui o novo objeto do tipo Cachorro a cachorro1





Polimorfismo

- Na POO, o tipo da variável de referência também pode ser uma superclasse.

```
Animal *meuCachorro = new Cachorro();
```

Variável do tipo Animal (superclasse)

Objeto Cachorro



- Isso define o conceito de **polimorfismo.**



Polimorfismo

- Exemplo: Criação de um array polimórfico

```
Animal *animais[6];

animais[0] = new Cachorro();
animais[1] = new Gato();
animais[2] = new Lobo();
animais[3] = new Leao();
animais[4] = new Hipopotamo();
animais[5] = new Animal();

for (int i = 0; i < 6; i++){
       animais[i]->fazerRuido();
       animais[i]->comer();
}
```



Sobrescrita de métodos

- Virtual:
  1. Um método **`virtual`** (superclasse) define que este será ligado dinamicamente, ou seja, de acordo com sua instanciação. Caso seja usado alocação dinâmica de memória. Usado em situações de sobrescrita de métodos e polimorfismo .
  2. Um método **`virtual puro`** (superclasse) deve obrigatoriamente ser sobrescrito pela classe filha (subclasse);
     1. A superclasse que tiver um método virtual puro será uma classe Abstrata e não poderá ser instanciada.



Polimorfismo - Virtual

- Colocar o **virtual**, na superclasse, antes do nome do método a ser sobrescrito. Não precisa do **virtual** na subclasse.

```
class Animal {
   public:
        virtual void fazerRuido(){
             cout << "Ruído Padrão\n";
        }
        virtual void comer(){
             cout << "Comer Padrão\n";
        }
        void dormir (){
             cout << "zzzz\n";
        }
        void passear(){
             cout << "Animal Passeando\n";
        }
};
```

Polimorfismo

- Podemos ter também atributos ou tipos de retorno polimórfico

```
class Veterinario{
   public:
   void vacinar(Animal *a){
       a->fazerRuido();
   }
};
```

```
int main(){
    Veterinario v = Veterinario();

    Cachorro c = Cachorro();
    Hipototamo h = Hipopotamo();
    v.vacinar(&c);
    v.vacinar(&h);
}
```



Polimorfismo

- Podemos ter também atributos ou tipos de retorno polimórfico

```
class Veterinario{
   public:
   void vacinar(Animal *a){
       a->fazerRuido();
   }
};
```

Parâmetro pode ser qualquer tipo de `Animal`

```
int main(){
    Veterinario v = Veterinario();

    Cachorro c = Cachorro();
    Hipototamo h = Hipopotamo();
    v.vacinar(&c);
    v.vacinar(&h);
}
```

Chamará o método `vacinar()` para Cachorro



Polimorfismo

- Podemos ter também atributos ou tipos de retorno polimórfico

```
class Veterinario{
   public:
   void vacinar(Animal *a){
       a->fazerRuido();
   }
};
```

Parâmetro pode ser qualquer tipo de Animal

```
int main(){
    Veterinario v = Veterinario();

    Cachorro c = Cachorro();
    Hipototamo h = Hipopotamo();
    v.vacinar(&c);
    v.vacinar(&h);
}
```

Podemos utilizar Alocação estática

Chamará o método vacinar() para Hipopotamo



Polimorfismo

- Portanto, se usarmos **argumentos polimórficos** (parâmetros com o tipo da superclasse)...

- podemos passar qualquer objeto subclasse na chamada (em tempo de execução...)
  - Ex: Argumento de `vacinar()` é do tipo `Animal`, então podemos passar qualquer subclasse (`Cachorro`, `Gato`, etc.) na chamada!



Polimorfismo

- Além disso, o código de `Veterinário` não precisa ser alterado se um novo tipo de subclasse for introduzido:
  - Criar uma subclasse `Tigre`

```
class Veterinario{
    public:
    void vacinar(Animal *a){
        a->fazerRuido();
    }
};
```

Posso vacinar() um Tigre (nova subclasse) ou qualquer outro animal novo sem modificar meu código

```
int main (){
    Veterinario v = Veterinario();

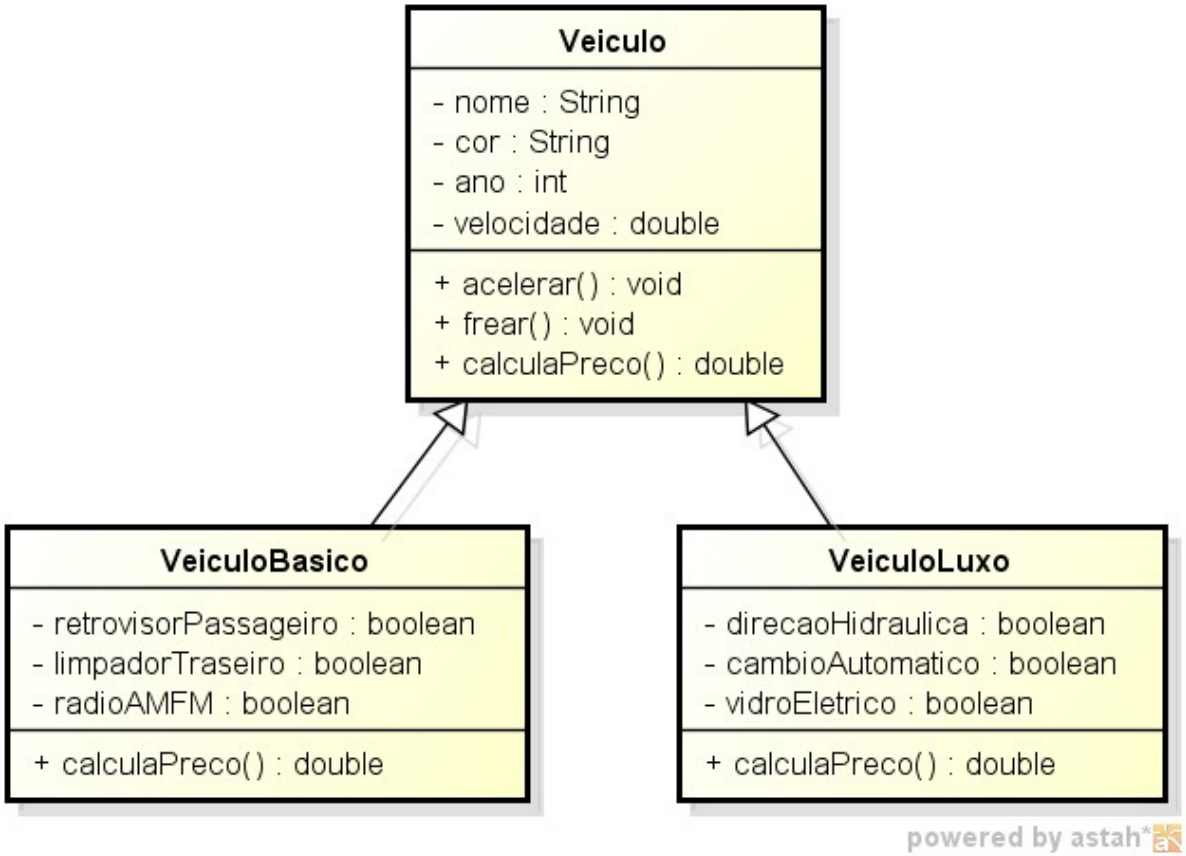
    Tigre t = Tigre();
    v.vacinar(&t);
}
```

vacinando o Tigre



Polimorfismo

- Exemplo2:





Polimorfismo

- Exemplo2:

```
class Concessionaria {
    public:
    void imprimePreco(Veiculo *v){
        cout << v->calculaPreco() << endl;
    }
};
```

Argumento polimórfico

Chamada polimórfica

```
int main(){
    Concessionaria c = Concessionaria();
    VeiculoBasico vb1 = VeiculoBasico();
    vb1.setRadioAMFM(true);

    VeiculoLuxo vL1 = VeiculoLuxo();
    vl1.setDirecaoHidraulica(true);

    c.imprimePreco(&vb1);
    c.imprimePreco(&vL1);
}
```



Polimorfismo

- Exemplo2:

```
class Concessionaria {
    public:
    void imprimePreco(Veiculo *v){
        cout << v->calculaPreco() << endl;
    }
};
```

Argumento polimórfico

Chamada polimórfica

```
int main(){
    Concessionaria c = Concessionaria();
    VeiculoBasico vb1 = VeiculoBasico();
    vb1.setRadioAMFM(true);

    VeiculoLuxo vL1 = VeiculoLuxo();
    vl1.setDirecaoHidraulica(true);

    c.imprimePreco(&vb1);
    c.imprimePreco(&vL1);
}
```

Chamada calculaPreco de VeiculoBasico



Polimorfismo

- Exemplo2:

```
class Concessionaria {
    public:
    void imprimePreco(Veiculo *v){
        cout << v->calculaPreco() << endl;
    }
};
```

Argumento polimórfico

Chamada polimórfica

```
int main(){
    Concessionaria c = Concessionaria();
    VeiculoBasico vb1 = VeiculoBasico();
    vb1.setRadioAMFM(true);

    VeiculoLuxo vL1 = VeiculoLuxo();
    vl1.setDirecaoHidraulica(true);

    c.imprimePreco(&vb1);
    c.imprimePreco(&vL1);
}
```

Chamada calculaPreco de VeiculoLuxo



Polimorfismo

- Exemplo3:
  - Conexão Wi-Fi, 4G/5G, Bluetooth
    - Métodos conectar, read/write



Polimorfismo

- Exercício:
  - Implemente uma trecho de um sistema de uma biblioteca. Crie um vetor de 5 Itens e instancie esses 5 Itens.

